CSA12 Computer Architecture

Assessment Tool 2 – Debugging & Optimisation Task

Q1: Signed Integer Addition Overflow

Bug: Adding +120 and +50 in 8-bit two's complement gives 170, which exceeds the range (-128 to +127). Binary: 01111000 + 00110010 = 10101010. Since two positive numbers produce a negative result, overflow occurs. Detect overflow by checking if carry into the MSB differs from carry out of the MSB or if operands have the same sign but the result has a different sign. Handle overflow by raising an overflow flag, displaying an error, or using a larger data type.

Q2: Ripple Carry Adder Delay

Bug: Each bit waits for the previous carry, creating long propagation delay. Root cause: sequential carry propagation. Optimization: replace with a Carry Look-Ahead Adder (CLA), which computes carry signals in parallel using generate and propagate logic. Result: much lower delay and faster arithmetic performance.

Q3: Booth's Multiplication

Bug: Incorrect results for negative operands. Possible causes: wrong bit-pair checking (Q0,Q-1), incorrect arithmetic right shift, or missing sign extension. Fix: follow Booth's rules correctly, preserve sign during arithmetic shifts, extend the sign bit properly, and verify each iteration with test cases involving positive and negative numbers.

Q4: Restoring vs Non-Restoring Division

Bug: Restoring division restores the previous value whenever subtraction fails, increasing execution time. Optimization: use the non-restoring division algorithm, which postpones correction until the next iteration instead of restoring every time. This reduces arithmetic operations and improves execution speed.

Q5: IEEE 754 Floating Point

Bug: Adding very small and very large numbers in single precision causes precision loss due to exponent alignment, normalization, and rounding. Optimization: use double precision (64-bit), rearrange calculations to add similar-sized values first, and reduce rounding errors using numerically stable algorithms.